

# **Active Directory SOC Home Lab**

*Threat Detection and Log Analysis in a Simulated Enterprise Environment*

**Tahmid Abtahee**

B.S. Cybersecurity Candidate, University of South Florida

June 2026

## Why I Built This

This project is an Active Directory SOC home lab built from scratch in Oracle VirtualBox: a domain controller, a Windows 10 endpoint, a Splunk SIEM, and a Kali attacker machine, wired together to simulate the kind of enterprise network a SOC analyst monitors every day.

The goal was not just to stand up the infrastructure but to use it. I ran attack techniques mapped to MITRE ATT&CK against the environment, captured the resulting activity in Splunk, and analyzed the detections to understand what each technique looks like at the log level. Building and attacking the same environment meant experiencing both sides of the problem: configuring Active Directory correctly and recognizing how it gets targeted.

## Tools and Technologies

- Oracle VirtualBox 7.x (hypervisor and virtual networking)
- Splunk Enterprise 10.4 (SIEM, log aggregation, detection)
- Splunk Universal Forwarder and Sysmon (Olaf Hartong config) for endpoint telemetry
- Windows Server 2022 with Active Directory Domain Services
- Kali Linux 2024, Crowbar v0.4.2 (RDP brute force), Atomic Red Team v2.3.0

## Lab Architecture

The lab runs on a single VirtualBox NAT network (ADproject, 192.168.10.0/24) with static IPs assigned to each machine to mirror how production networks are configured.

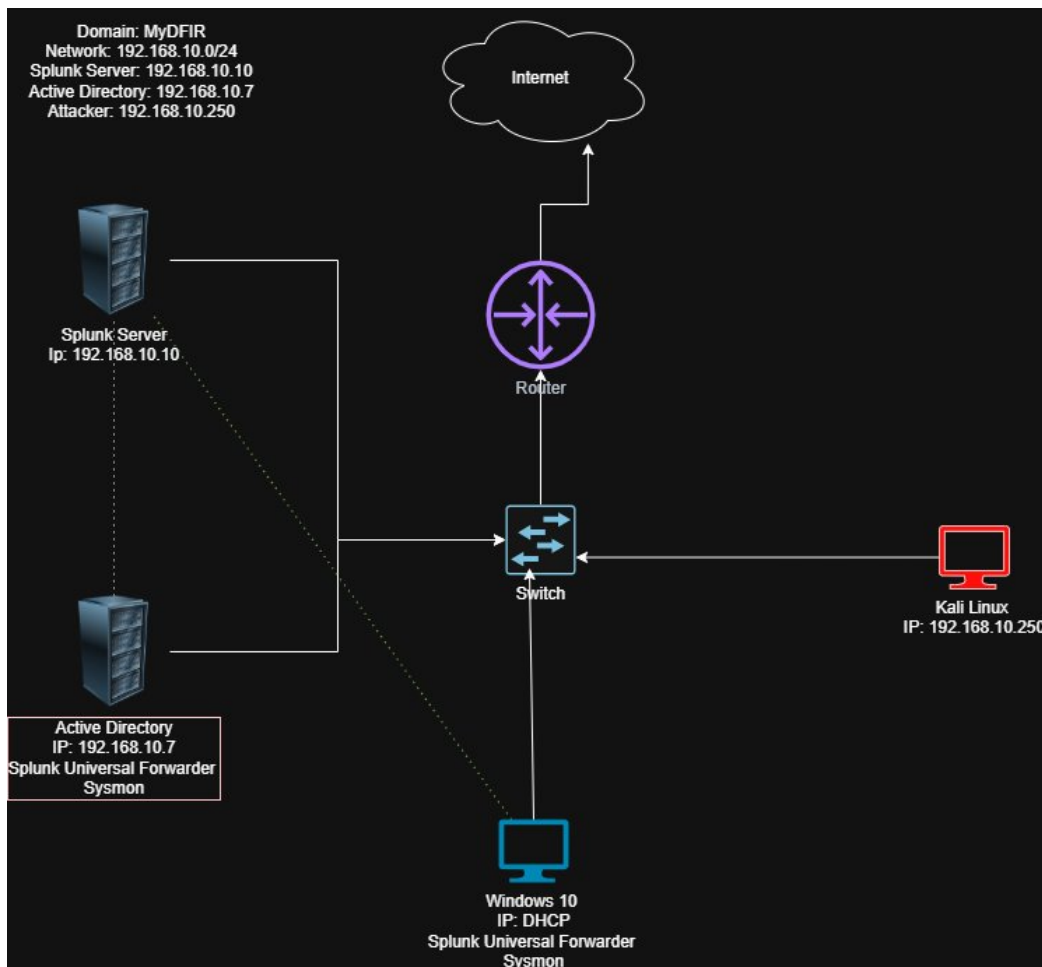


Figure 1. Lab topology: four VMs, their roles, IP assignments, and installed agents.

| Operating System    | Role                       | IP Address     | Key Tools                    |
|---------------------|----------------------------|----------------|------------------------------|
| Ubuntu Server 22.04 | Splunk SIEM                | 192.168.10.10  | Splunk Enterprise 10.4       |
| Windows Server 2022 | Domain Controller (ADDC01) | 192.168.10.7   | Sysmon + Universal Forwarder |
| Windows 10 Pro      | Target Endpoint            | 192.168.10.100 | Sysmon + Universal Forwarder |
| Kali Linux 2024     | Attacker Machine           | 192.168.10.250 | Crowbar, Atomic Red Team     |

Table 1. Virtual machine inventory.

Splunk was deployed headless on Ubuntu, and the Universal Forwarder plus Sysmon were installed on both Windows machines so activity on the domain controller and the endpoint were both visible to the SIEM.

## Environment Setup

### What I Did

Splunk Enterprise 10.4 was installed on Ubuntu Server via CLI and confirmed reachable at 192.168.10.10:8000. A dedicated index named endpoint was created to receive all forwarded logs, separating this data from Splunk's default index the way production deployments segregate log sources by type.

Sysmon was installed on both Windows machines using Olaf Hartong's community configuration, which enriches raw event codes with process hashes, parent process context, and network connection data, well beyond what default Windows logging provides. A custom inputs.conf was deployed to the Universal Forwarder's local directory (so it persists across updates) to route Application, Security, System, and Sysmon Operational logs into the endpoint index:

```
[WinEventLog://Application]
index = endpoint
disabled = false
[WinEventLog://Security]
index = endpoint
disabled = false
[WinEventLog://System]
index = endpoint
disabled = false
[WinEventLog://Microsoft-Windows-Sysmon/Operational]
index = endpoint
disabled = false
renderXml = true
source = XmlWinEventLog:Microsoft-Windows-Sysmon/Operational
```

inputs.conf deployed to C:\Program Files\SplunkUniversalForwarder\etc\system\local\ on both Windows machines.

The renderXml and source fields matter: they preserve the full XML structure of Sysmon events, so Splunk parses the enriched fields (process names, hashes, technique IDs) instead of returning raw binary data.

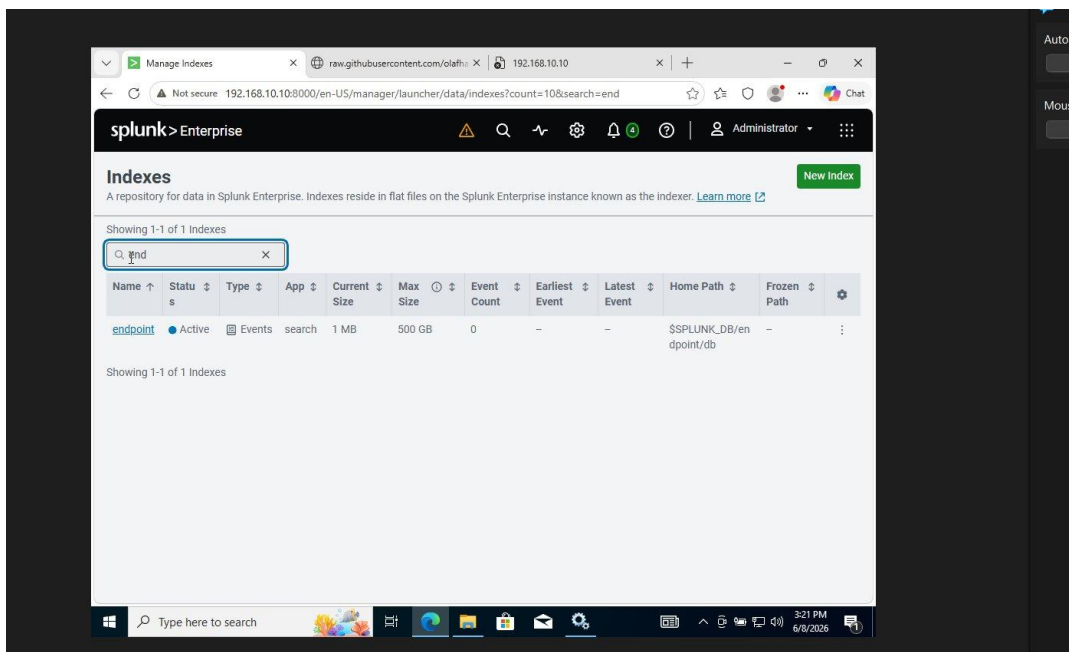


Figure 2. The endpoint index in Splunk confirming active status and receipt of logs from both Windows machines.

## Active Directory Configuration

### What I Did

Windows Server 2022 was promoted to a domain controller for a new forest, mydfir.local, via the AD DS Configuration Wizard. Two Organizational Units, IT and HR, were created to reflect a realistic administrative structure, and a domain user account was provisioned to serve as a target identity for the brute force simulation. The Windows 10 machine was then joined to the domain, and both domain accounts were granted Remote Desktop access in preparation for the attack.

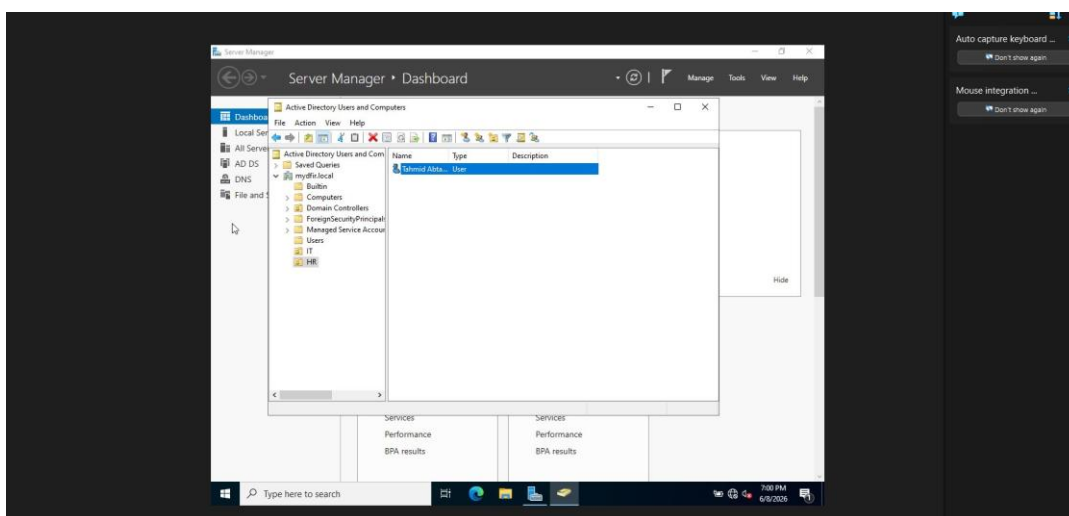


Figure 3. Active Directory Users and Computers showing the mydfir.local domain with IT and HR OUs and a provisioned domain user.

## Attack Simulations

Two categories of attacks were run from the Kali attacker machine (192.168.10.250) against the Windows 10 target (192.168.10.100), with the domain controller (192.168.10.7) monitored throughout for correlated events: a credential brute force over RDP, and MITRE ATT&CK technique simulations via Atomic Red Team.

## 5.1 Credential Brute Force via RDP (Crowbar)

**What I Did:** A targeted password list was built from the first 20 entries of rockyou.txt, with the correct password appended to the end, simulating an attacker working through a common wordlist before reaching valid credentials. Crowbar v0.4.2 then ran the brute force against port 3389 on the target.

```
(kali@kali)-[~/Desktop/ad-project]
$ head -n 20 rockyou.txt > passwords.txt

(kali@kali)-[~/Desktop/ad-project]
$ ls
passwords.txt  rockyou.txt

(kali@kali)-[~/Desktop/ad-project]
$ cat passwords.txt
123456
12345
123456789
password
iloveyou
princess
1234567
rockyou
12345678
abc123
nicole
daniel
babygirl
monkey
lovely
jessica
654321
michael
ashley
qwerty

(kali@kali)-[~/Desktop/ad-project]
$ nano passwords.txt

(kali@kali)-[~/Desktop/ad-project]
```

Figure 4. passwords.txt, built from rockyou.txt with the correct password appended as the final entry.

```
(kali@kali)-[~/Desktop/ad-project]
$ ping 192.168.10.10
PING 192.168.10.10 (192.168.10.10) 56(84) bytes of data.
rom 192.168.10.250 icmp_seq=1 Destination Host Unreachable
rom 192.168.10.250 icmp_seq=2 Destination Host Unreachable
rom 192.168.10.250 icmp_seq=3 Destination Host Unreachable
C
 192.168.10.10 ping statistics ---
  packets transmitted, 0 received, +3 errors, 100% packet loss, time 5098ms
 iperf 3

(kali@kali)-[~/Desktop/ad-project]
$ ping 192.168.10.10
PING 192.168.10.10 (192.168.10.10) 56(84) bytes of data.
 4 bytes from 192.168.10.10: icmp_seq=1 ttl=64 time=9.85 ms
 4 bytes from 192.168.10.10: icmp_seq=2 ttl=64 time=2.24 ms
 4 bytes from 192.168.10.10: icmp_seq=3 ttl=64 time=5.54 ms
 4 bytes from 192.168.10.10: icmp_seq=4 ttl=64 time=2.63 ms
 4 bytes from 192.168.10.10: icmp_seq=5 ttl=64 time=3.76 ms
 4 bytes from 192.168.10.10: icmp_seq=6 ttl=64 time=2.30 ms
 4 bytes from 192.168.10.10: icmp_seq=7 ttl=64 time=3.10 ms
 4 bytes from 192.168.10.10: icmp_seq=8 ttl=64 time=3.54 ms
C
 192.168.10.10 ping statistics ---
  packets transmitted: 8 received, 0% packet loss, time 7074ms
 tt min/avg/max/mdev = 2.238/4.119/9.846/2.381 ms

(kali@kali)-[~/Desktop/ad-project]
$ crowbar -b rdp -u taho -C passwords.txt -s 192.168.10.10/32
026-06-08 21:36:50 START
026-06-08 21:36:50 Crowbar v0.4.2
026-06-08 21:36:50 Trying 192.168.10.10:3389
026-06-08 21:36:51 STOP
026-06-08 21:36:51 No results found ...

(kali@kali)-[~/Desktop/ad-project]
$ crowbar -b rdp -u saiba -C passwords.txt -s 192.168.10.100/32
026-06-08 21:36:58 START
026-06-08 21:36:58 Crowbar v0.4.2
026-06-08 21:36:58 Trying 192.168.10.100:3389
026-06-08 21:36:59 STOP
026-06-08 21:36:59 No results found ...

(kali@kali)-[~/Desktop/ad-project]
$ crowbar -b rdp -u taho -C passwords.txt -s 192.168.10.10/32
026-06-08 21:37:11 START
026-06-08 21:37:11 Crowbar v0.4.2
026-06-08 21:37:11 Trying 192.168.10.10:3389
026-06-08 21:37:11 STOP
026-06-08 21:37:11 No results found ...
```

Figure 5. Crowbar v0.4.2 executing the RDP brute force against 192.168.10.100:3389.

## 5.2 MITRE ATT&CK Simulations (Atomic Red Team)

**T1136.001, Create Local Account (Persistence):** Atomic Red Team created a local administrator account (NewLocalUser), confirmed the creation, then deleted it, simulating a backdoor account an attacker would use to maintain access independent of their original entry point.

**T1059.001, PowerShell Execution (Mimikatz and BloodHound):** Atomic Red Team attempted to run Mimikatz and BloodHound via PowerShell. Windows Defender blocked the Mimikatz execution and generated a threat alert. Even blocked, the attempt still created process creation events that Splunk captured.

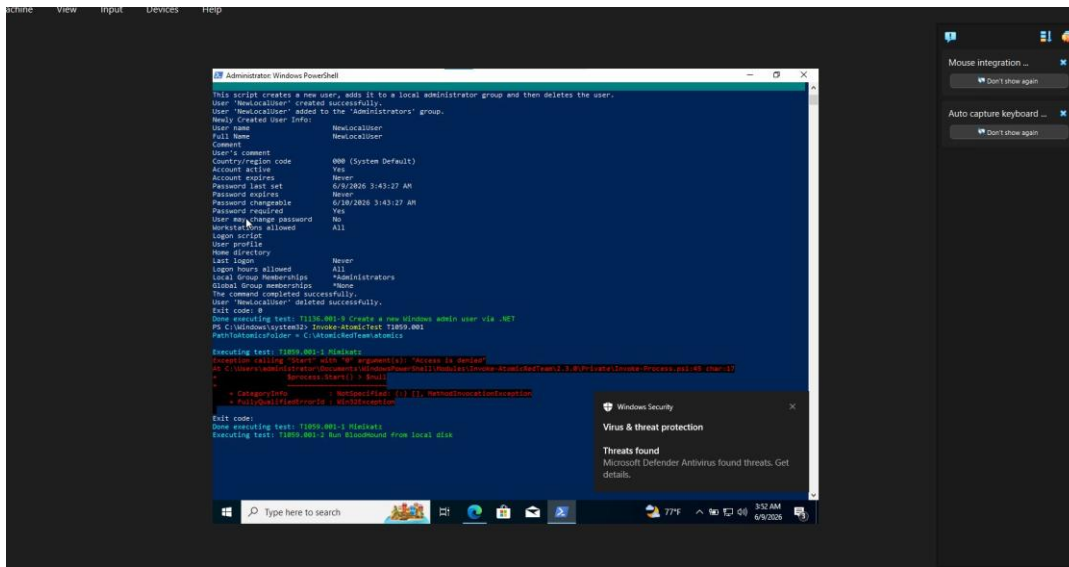


Figure 6. T1136.001 executing (NewLocalUser created, added to Administrators, then deleted) alongside a simultaneous Windows Defender alert from the T1059.001 Mimikatz attempt.

## Detection and Analysis

All activity was detected in Splunk against the endpoint index across three event codes.

### 6.1 Brute Force via EventCode 4625

```
index=endpoint EventCode=4625
```

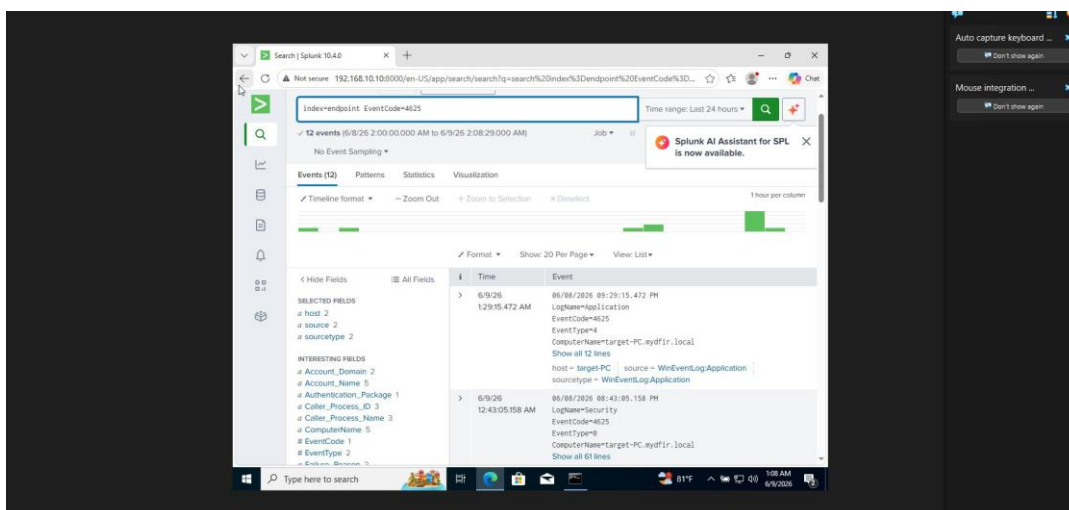


Figure 7. 12 failed logon events on target-PC.mydfir.local, clustered in a narrow time window.

All 12 failures were clustered within an extremely narrow window, the behavioral signature of an automated tool rather than a human mistyping a password. This clustering is the same pattern that triggers account lockout policies and SIEM brute force alerting in production.

6.2 Persistence via EventCode 4720

```
index=endpoint EventCode=4720
```

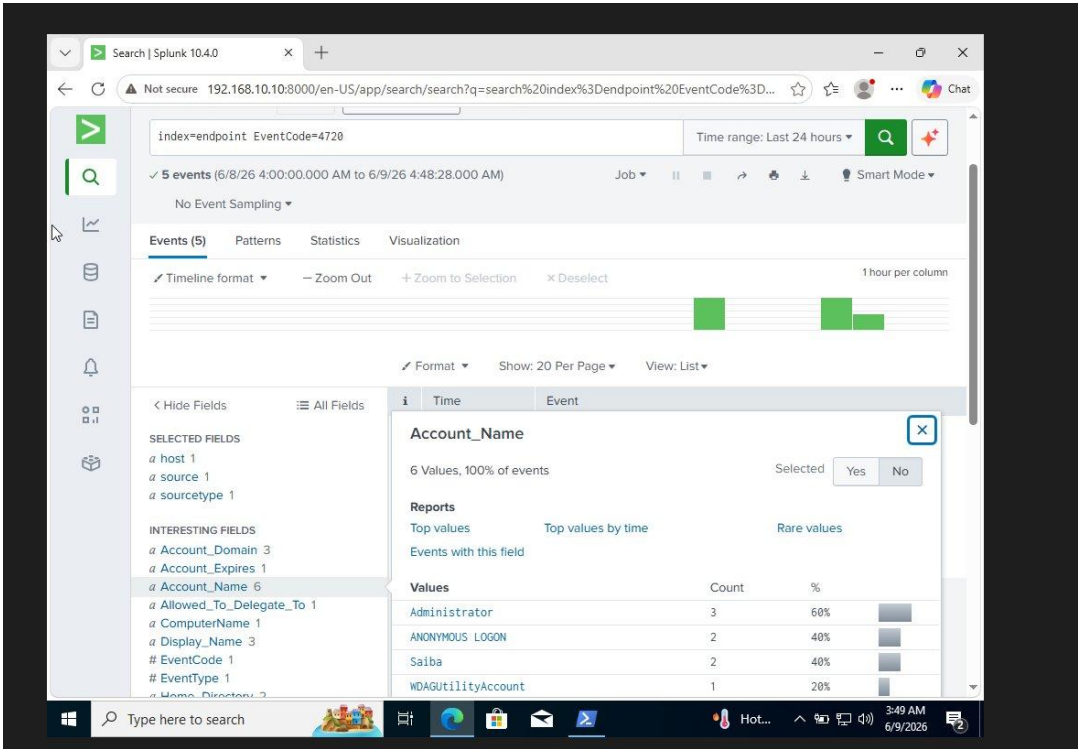


Figure 8. 5 events for EventCode=4720, with NewLocalUser visible in the Account\_Name breakdown.

The account creation was logged and retained even though the account was deleted immediately afterward; the deletion does not erase the creation record. A follow-up free-text search for the username NewLocalUser (without specifying an event code) returned zero results, confirming that detections need to be anchored to the correct field, not a keyword guess.

## 6.3 Process Creation via EventCode 4688

```
index=endpoint EventCode=4688
```

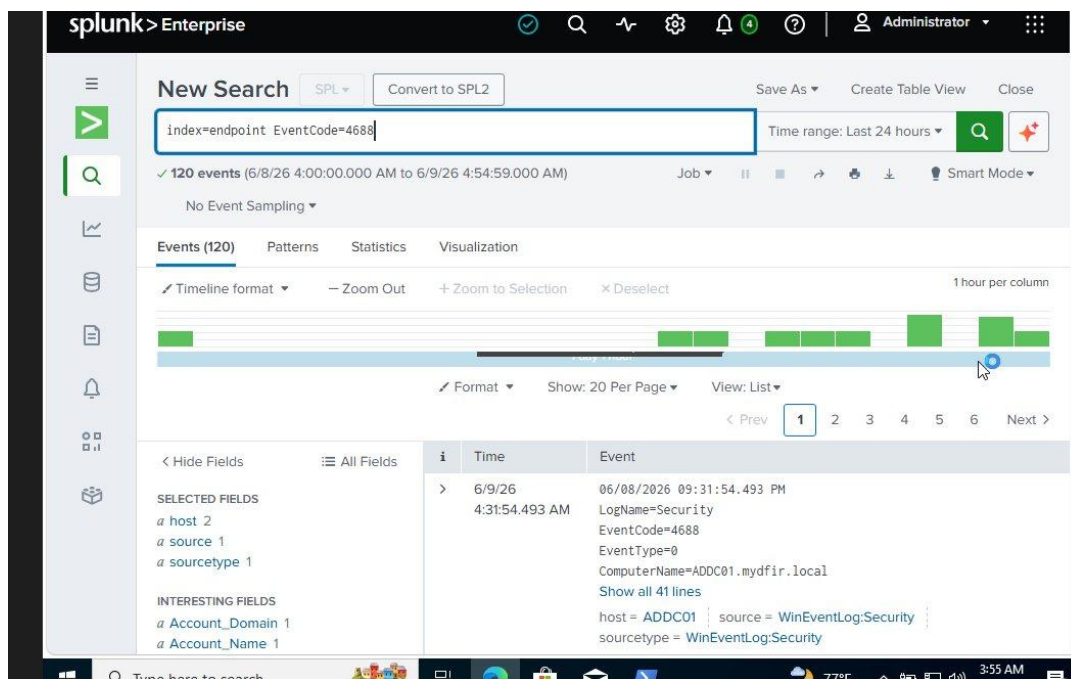


Figure 9. 120 process creation events across the testing period, including activity from the domain controller.

The 120 events include the Mimikatz attempt that Defender blocked. A blocked attack still leaves forensic evidence: process creation logging does not depend on the attack succeeding, which matters in environments where endpoint protection is disabled or tampered with.

## Detection Logic and SPL Query Reference

Each detection below is documented with inline comments explaining the reasoning behind every clause, the way detection rules are reviewed and audited in production SOC environments.

### 7.1 Brute Force Detection (T1110)

```
index=endpoint EventCode=4625
| stats count by src_ip Account_Name host
| where count > 10
| sort - count
| rename src_ip AS "Source IP", Account_Name AS "Targeted Account", count AS "Failed Attempts"

// Aggregates failed logons by source, account, and host.
// Threshold of 10 aligns with common account-lockout policy
// thresholds and filters out ordinary user typos.
```

### 7.2 Persistence Detection (T1136.001)

```
index=endpoint EventCode=4720
| table _time host Account_Name SAM Account_Name Subject Account_Name
| rename Account_Name AS "New Account", Subject_Account_Name AS "Created By",
  _time AS "Time", host AS "Host"
| sort - Time
```

```
// Subject_Account_Name reveals WHO created the account.
// An unexpected creator (non-IT, non-admin) is a high-fidelity
// indicator of compromise.
```

### 7.3 Execution Detection (T1059.001)

```
index=endpoint EventCode=4688
| stats count by host New_Process_Name Creator_Process_Name Account_Name
| where like(Creator_Process_Name, "%powershell%")
  OR like(Creator_Process_Name, "%cmd.exe%")
| where count > 5
| sort - count
| rename New_Process_Name AS "Spawned Process", Creator_Process_Name AS "Parent Process",
  Account_Name AS "User", count AS "Event Count"

// Parent-child process relationships are the primary signal:
// Word.exe spawning powershell.exe is malicious,
// explorer.exe spawning notepad.exe is not.
```

### Detection Summary

| Detection   | MITRE Technique | Event Code | Key Field            | Threshold Logic                      |
|-------------|-----------------|------------|----------------------|--------------------------------------|
| Brute Force | T1110           | 4625       | src_ip count         | count > 10 failures                  |
| Persistence | T1136.001       | 4720       | Subject_Account_Name | Any new account creation             |
| Execution   | T1059.001       | 4688       | Creator_Process_Name | PowerShell/cmd.exe parent > 5 events |

### Challenges and Resolutions

| Issue                                     | Root Cause                                                                                                                                        | Resolution                                                                    |
|-------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------|
| Domain controller performance degradation | VM boot order prioritized network (PXE) boot over the hard disk, so every startup timed out on a PXE attempt before falling back to disk.         | Reordered the boot sequence in VirtualBox to prioritize the hard disk first.  |
| Crowbar targeted the wrong host           | The brute force was first run against 192.168.10.10 (the Splunk server), which has no RDP enabled, instead of the intended 192.168.10.100 target. | Verified connectivity with ping and re-ran the attack against the correct IP. |

### Lessons Learned

1. **Temporal clustering is the clearest signal of automated brute force.** Twelve failed logons landing in a sub-second window from one source is not something a human produces by mistyping a password. This pattern is the basis for both account lockout policies and automated SIEM alerting in production.
2. **Account creation events outlive the account.** EventCode 4720 logged the NewLocalUser persistence account before it was deleted. Detection of a persistence mechanism does not depend on the account still existing at investigation time.
3. **A blocked attack is not an undetected one.** Windows Defender stopped the Mimikatz execution, but EventCode 4688 still logged the attempt. SIEM process monitoring provides a detection layer that does not depend on endpoint protection succeeding or even being enabled.

4. **Detection specificity matters.** A free-text search for the username NewLocalUser returned zero results; the same account only surfaced once the search was anchored to EventCode 4720. Detections have to be built on the correct field, not a keyword guess.
5. **Telemetry quality is a force multiplier.** Olaf Hartong's Sysmon configuration added process hashes, parent-child context, and command-line arguments that default Windows logging does not provide, meaningfully cutting the analysis effort needed to tell a benign process from a malicious one.
6. **Infrastructure problems are part of the job.** Both the boot-order issue and the mistargeted brute force were resolved through the same process: verify assumptions, isolate the variable, and confirm the fix, rather than reinstalling or guessing.

## Conclusion

This lab answers a specific question: what does it actually take to detect and defend an Active Directory environment under attack? Building the infrastructure, running real attack tooling against it, and tracing each technique back to its event code and SPL query gave that question a concrete answer grounded in real telemetry rather than theory. Active Directory remains the authentication backbone of most enterprise networks, and the ability to configure it, monitor it, and recognize when it is being targeted is directly transferable to production SOC and detection engineering work.